SD242 – Verificação de Sistemas Digitais.

Autores	
Anderson Max Almeida e Porfirio	

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

Aula 22
Transaction-Level Modeling (TLM)

Tópicos

- Introdução
- TLM – Transaction Level Modeling
- UVC
- TLM, TLM-1 e TLM-2.0
- TLM-1 no UVM
- Interfaces TLM
- Comunicação TLM Básica (1:1 / blocking)
- Comunicação TLM Básica
- Uvm_tlm_fifo
- Bloqueante vs. Não-bloqueante
- Conectando componentes
- Analysis Ports
- Analysis Export
- Conexão e Execução
- Uvm_analysis_fifo

Introdução

- Uma das chaves para a produtividade na verificação é pensar no problema em um certo nível de abstração.
- Ao verificar um DUT que lida com pacotes, processa instruções, ou executa outras funcionalidades, você deve criar um ambiente de verificação que suporte o nível de abstração apropriado.

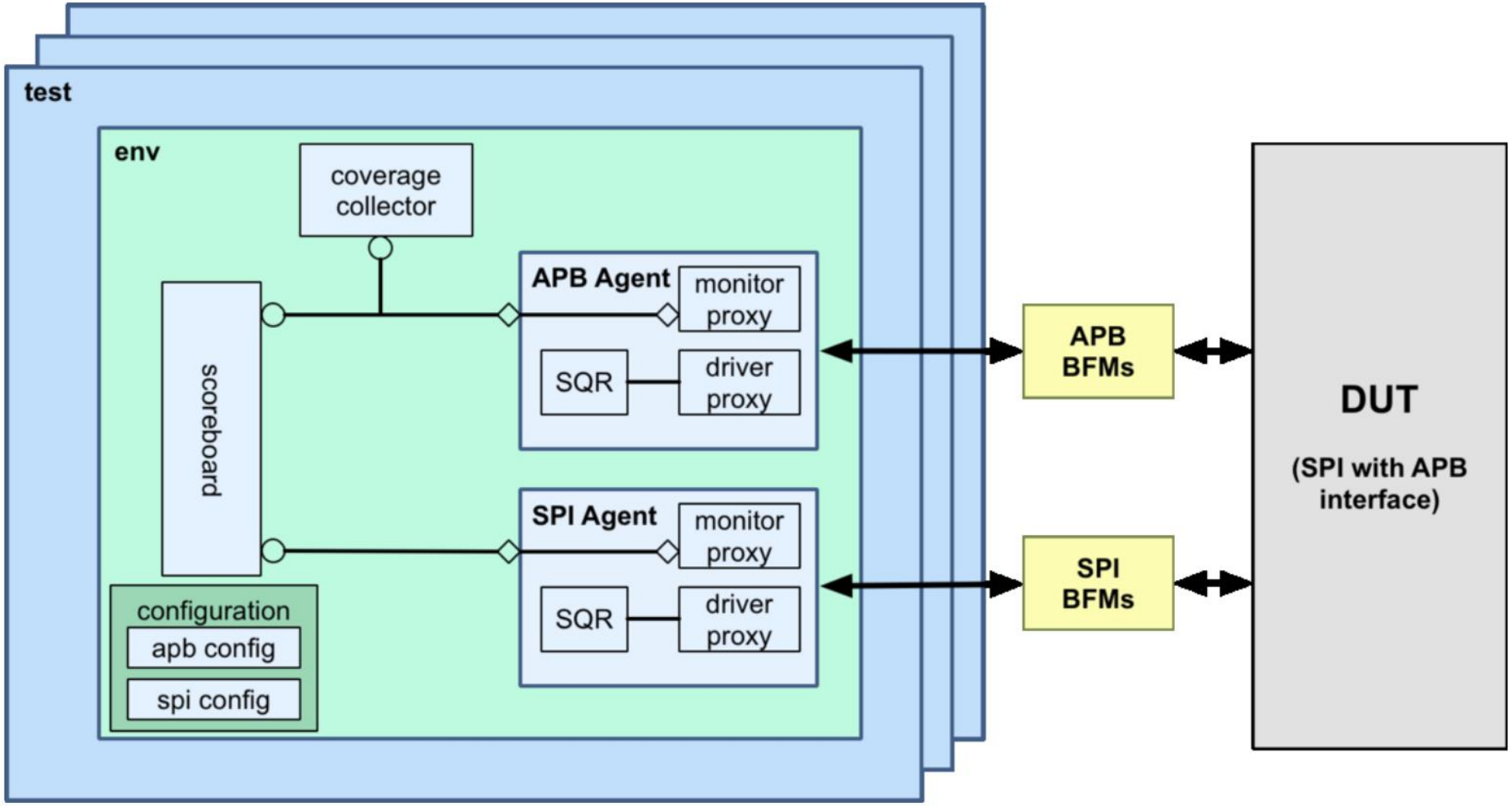
- Interface Real vs. Tarefas de Verificação:
  - A interface real para o DUT é representada pela atividade em nível de sinal.
  - Porém, é necessário gerenciar a maioria das tarefas de verificação (como geração de estímulo e coleta de dados de cobertura) no nível de transação.
  - O nível de transação é a forma natural como os engenheiros tendem a pensar na atividade de um sistema.

Introdução

- TLM-1 e TLM-2.0
  - São dois sistemas de modelagem TLM que foram desenvolvidos como padrões industriais (originalmente no SystemC pela OSCI).
- TLM-1.0 (2005)
  - É um sistema de passagem de mensagens (message passing system). As interfaces são untimed ou dependem do target para o timing. Não fornecem anotações de timing explícitas.
- TLM-2.0 (2009)
  - Principalmente projetado para modelagem de alto desempenho de sistemas baseados em barramento memory-mapped.

- Implementação no UVM
  - Um subconjunto de ambas as facilidades (TLM-1 e TLM-2.0) foi implementado em SystemVerilog e está disponível como parte do padrão UVM.

TLM - Transaction Level Modeling



TLM

- O UVM fornece um conjunto de interfaces que são canais de comunicação em nível de transação que você pode usar para conectar componentes neste nível.

- O uso de interfaces TLM:
  - Isola cada componente de alterações em outros componentes.
  - Promove o reuso, permitindo que qualquer componente seja trocado por outro, desde que tenham as mesmas interfaces.

TLM

- Esse conceito permite que ambientes UVM sejam montados com um modelo em nível de transação do DUT e que o ambiente seja reutilizado à medida que o design é refinado para RTL.

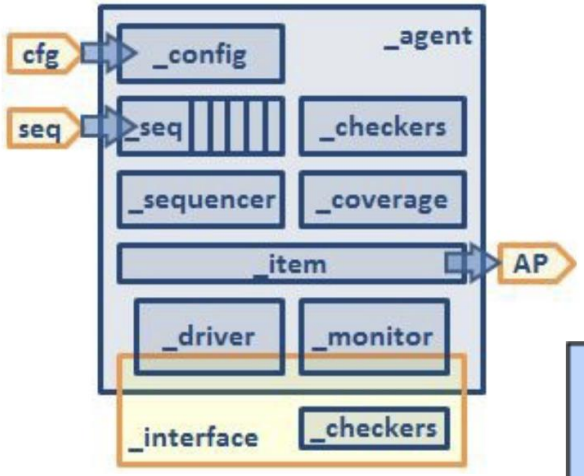
- Basta substituir o modelo em nível de transação por uma camada fina de componentes compatíveis para converter entre a atividade em nível de transação e a atividade em nível de pino no DUT.

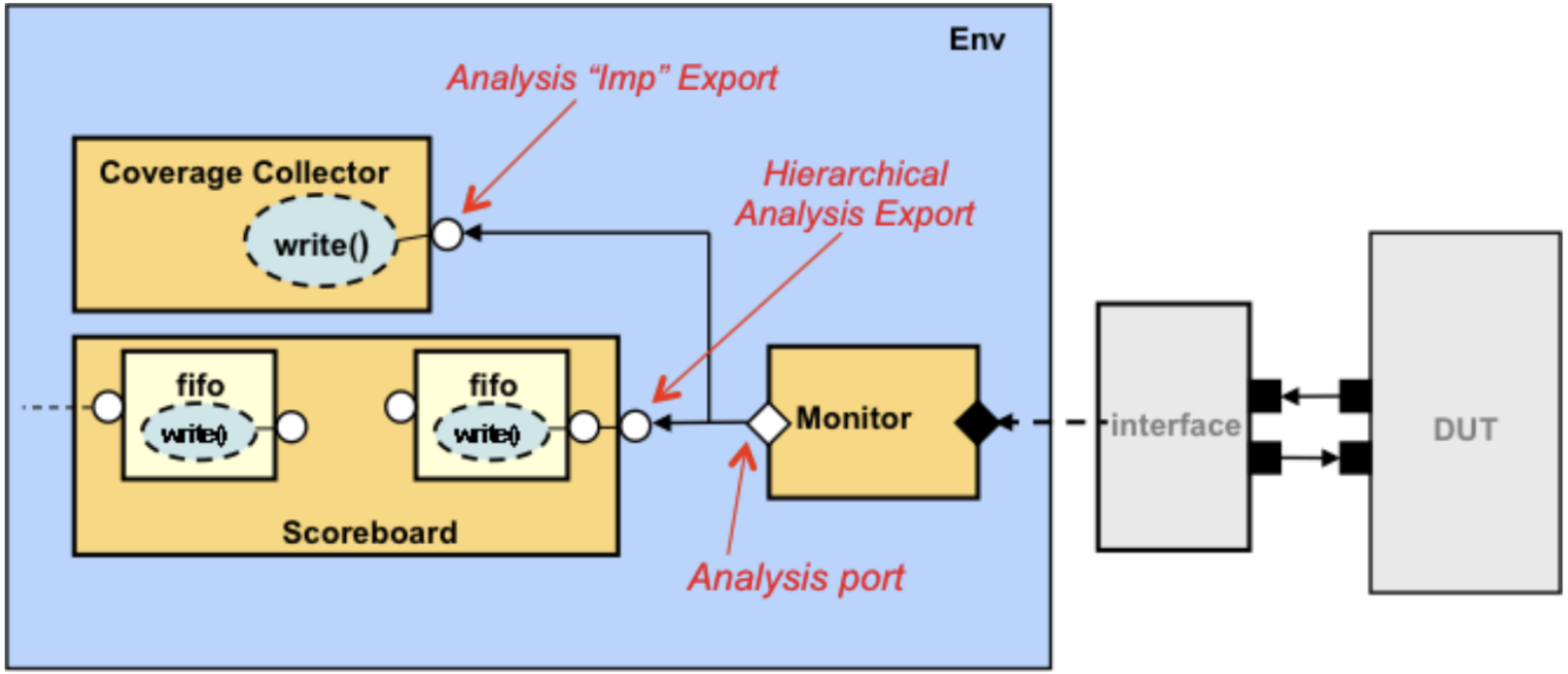
- Fornece a base para encapsular facilmente componentes em componentes de verificação reutilizáveis UVC (universal verification components).

UVC (Verification Component)

- A arquitetura de uma de protocolo UVC inclui estes elementos essenciais.
- **Driver**: Converte transações abstratas do sequenciador em sinais no nível dos pinos.
- **Monitor**: Observa os sinais dos pinos, reconstrói transações e as envia para análise.
- **Interface**: Contém as conexões lógicas dos pinos e opcional métodos (BFM) para interagir com eles.
- **Item/Transação**: Classe base que modela a transação, com restrições e aspectos comuns.
- **Sequenciador**: Alimenta o driver com sequências de estímulo.
- **Sequências** "API": Biblioteca de sequências básicas para construir estímulos complexos.
- **Configuração**: Configuração do protocolo e do ambiente, com a interface virtual.
- **Elementos opcionais**:
  - Objeto de Cobertura (Coverage): Coleta métricas de cobertura das transações.
  - Scoreboard (Checker): Valida regras de protocolo de alto nível (quando necessário).
  - Agente: Componente que pode agrupar os elementos acima.
- Todos os componentes são agrupados em um (package), em uma hierarquia organizada.

UVC


cfg
_config
_agent
seq
_seq
_checkers
_sequencer
_coverage
_item
AP
_driver
_monitor
_interface
_checkers


Env
Analysis "Imp" Export
Coverage Collector
write()
Hierarchical
Analysis Export
fifo
write()
fifo
write()
Monitor
Scoreboard
Analysis port
interface
DUT

TLM, TLM-1 e TLM-2.0

- TLM (Transaction-Level Modeling)
  - É um estilo de modelagem para a construção de modelos altamente abstratos de componentes e sistemas.
  - Baseia-se em transações (objetos que contêm dados arbitrários e específicos do protocolo) para representar de forma abstrata a atividade de nível inferior.

- Abstrações Comuns em Nível de Transação
  - O TLM refere-se a uma família de níveis de abstração que começam com a modelagem ciclo-precisa (cycle-accurate), e se estende a o níveis mais abstrato.

- As abstrações comuns incluem
  - cycle-accurate, approximately-timed, loosely-timed, untimed, e token-level.

TLM-1 no UVM

- Em UVM, uma transação é um objeto de classe que inclui qualquer informação necessária para modelar uma unidade de comunicação entre dois componentes.

```
class simple_trans extends uvm_sequence_item;
    rand data_t data;
    rand addr_t addr;
    rand enum {WRITE,READ} kind;
    constraint c1 { addr < 16'h2000; }
  ...
Endclass
```

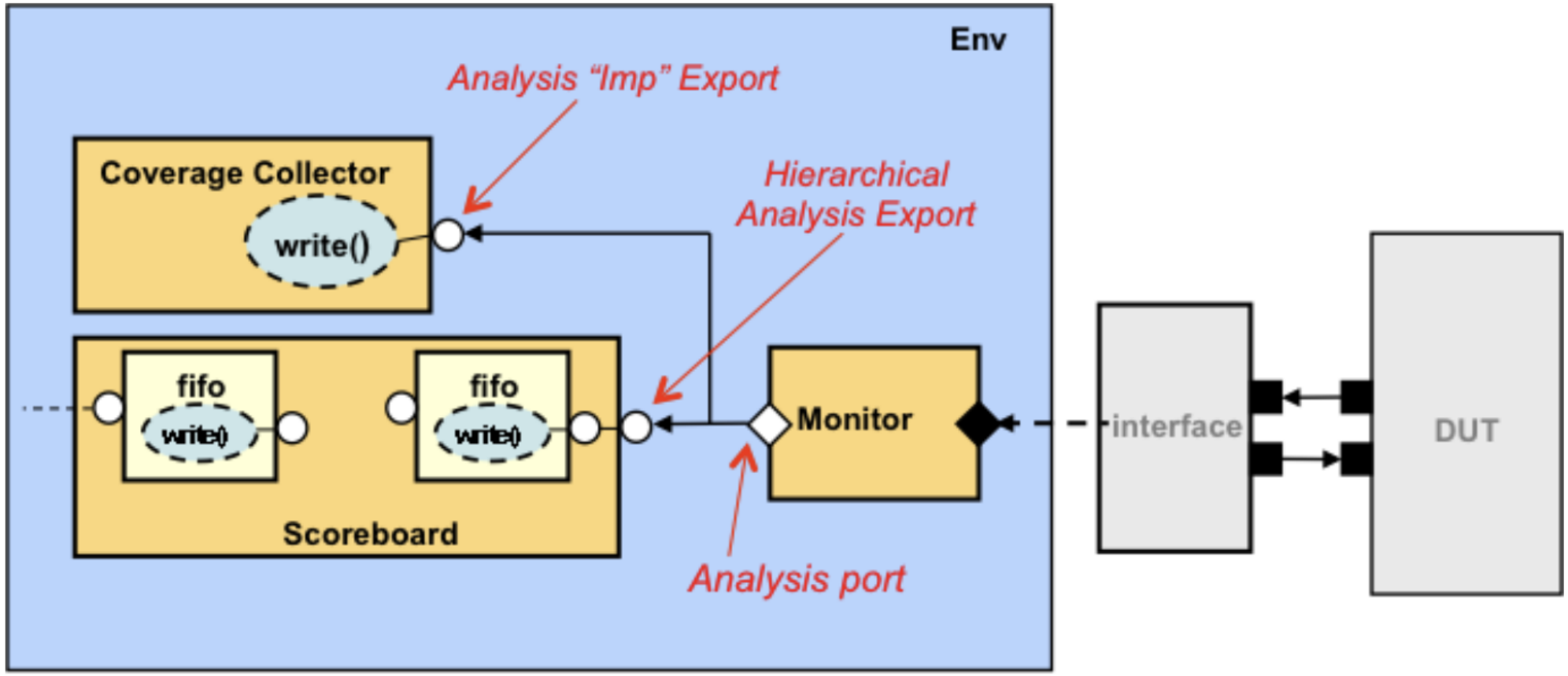
  - O objeto de transação inclui variáveis, constraints, e outros campos e métodos necessários para gerar e operar a transação.
  - A quantidade e o detalhe das informações encapsuladas em uma transação são uma indicação do nível de abstração do modelo.
  - As transações podem ser compostas, decompostas, estendidas, em camadas e manipuladas de outras formas para modelar a comunicação necessária em qualquer nível de abstração.

Interfaces TLM

- Definem um conjunto de métodos que usam objetos de transação como argumentos.
- Porta (*Port*) e Exportação (*Export*)
  - Um Port (TLM port) define o conjunto de métodos (a API) a ser usado para uma conexão específica.
  - Um Export (TLM export) fornece a implementação desses métodos.

- Conexão
  - Conectar um port a um export permite que a implementação seja executada quando o método do port é chamado.

Interfaces TLM



Comunicação TLM Básica (1:1 / blocking)

- Produtor (producer)
  - Gera transações e as envia para fora através de seu put_port.
  - A chamada put() no produtor bloqueará até que a implementação put do consumidor seja concluída (método blocking).
  - O produtor continua a funcionar exatamente da mesma forma se o consumidor for substituído por outro componente que também implemente put (modularidade).

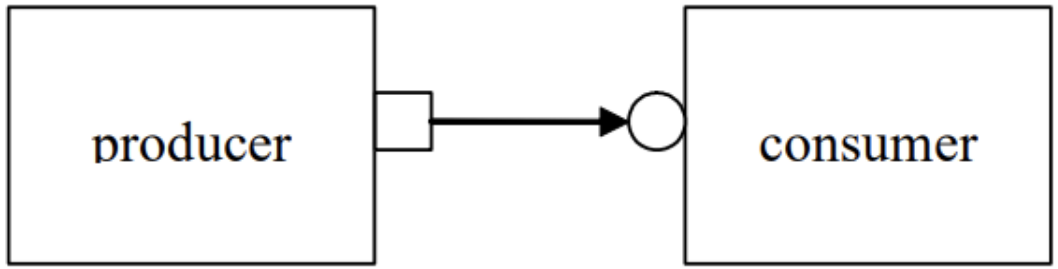


Figure 4—Single Producer/Consumer

Comunicação TLM Básica

- Consumidor (consumer)
  - Fornece a implementação real da chamada put().
  - Solicita transações do produtor através de sua get_port.
  - A chamada get() do consumidor bloqueará até que o método do produtor seja concluído (método blocking).
  - A direção do fluxo de dados (do produtor para o consumidor) é a mesma em ambos os exemplos (put e get).

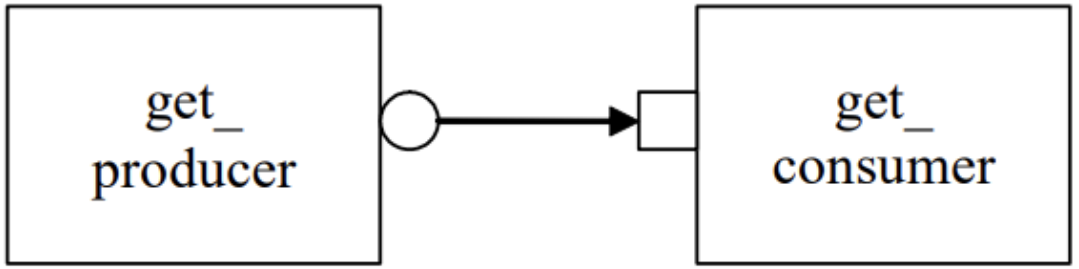


Figure 5—Consumer gets from Producer

uvm_tlm_fifo

- Operação Independente
  - Os componentes precisam operar independentemente (produtor cria transações em um processo enquanto o consumidor opera em outro).
- O UVM fornece o canal uvm_tlm_fifo para facilitar essa comunicação.
  - O produtor put a transação no uvm_tlm_fifo.
  - O consumidor get a transação do fifo de forma independente.

- Put() bloqueia se o fifo estiver cheio, caso contrário, insere o objeto e retorna imediatamente (comportamento não bloqueante se houver espaço).
- Get() retorna uma transação estiver disponível (e a remove), caso contrário, bloqueia até que uma transação esteja disponível.
- peek() retorna uma cópia da transação disponível sem removê-la.

uvm_tlm_fifo

- O FIFO é especial porque ele implementa o put (para receber do produtor) e o get (para fornecer ao consumidor), permitindo que os dois componentes rodem em processos independentes sem bloquear um ao outro (até que o FIFO esteja cheio ou vazio).

- Neste caso, o FIFO atua como o intermediário que implementa ambos os lados:
- Produtor -> FIFO: O Produtor usa um put_port para *put* a transação no FIFO.
- FIFO -> Consumidor: O Consumidor usa um get_port para *get* a transação do FIFO.

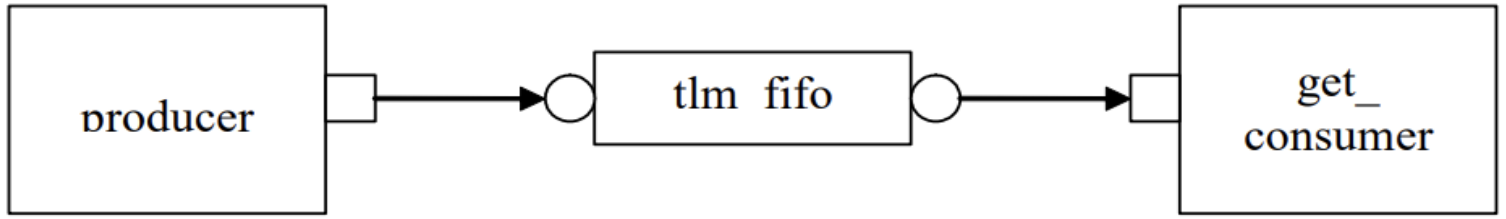


Figure 6—Using a uvm_tlm_fifo

Bloqueante vs. Não-Bloqueante (Blocking versus Nonblocking)

- Comunicação Bloqueante (tasks put, get)
  - Não é permitido falhar ou alterar o fluxo de controle.
  - Elas simplesmente esperam até que a solicitação seja satisfeita.
  - Em um sistema temporizado, isso significa que o tempo pode passar entre o início da chamada e seu retorno.

- Comunicação Não-Bloqueante (try_put, try_get, try_peek)
  - Uma chamada não-bloqueante (nonblocking) retorna imediatamente.
  - Retorna no mesmo ciclo delta em que foi emitida, ou seja, sem consumir tempo, nem mesmo um único ciclo delta.
  - Em UVM, chamadas não-bloqueantes são modeladas como funções (function).

Bloqueante vs. Não-Bloqueante (Blocking versus Nonblocking)

- try_get

```
// ...
task run;
for(int i=0; i<10; i++)
  if(get_port.try_get(t))
    //Faz algo com t.
...
endtask
// ...
```

- Retorno
  - Se uma transação existir, ela será retornada no argumento e a função retornará TRUE.
  - Se nenhuma transação existir, a função retornará FALSE.
  - O mesmo princípio se aplica a try_peek() e try_put().

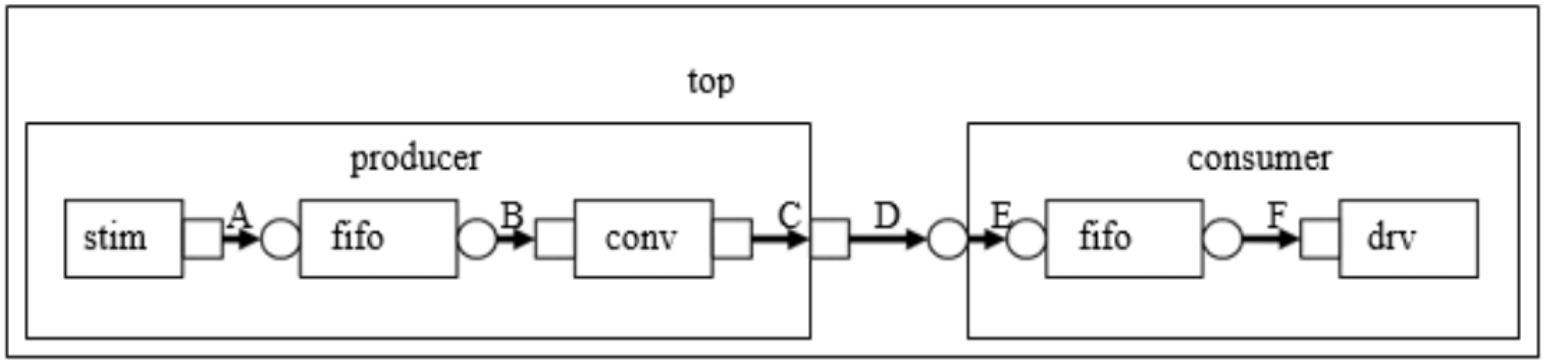
Conectando Componentes

- A conexão entre portas (ports) e exports é realizada através do método connect() no componente pai (parent).
- A série de chamadas connect() cria uma rede de conexões (netlist) que pode ser complexa, com várias etapas (hierárquica) ou direta entre dois pontos (peer-to-peer)
  - Quando um componente chama put_port.put(t);
  - a conexão significa que ele realmente chama target.put_export.put(t);
  - onde target(alvo) é o componente conectado.

- Conexões Peer-to-Peer
  - Conexões entre componentes no mesmo nível de hierarquia.
  - Regra: Portas são sempre conectadas a *exports*.
  - Todas as chamadas connect() entre componentes são feitas no método connect_phase do componente pai.

Conectando Componentes

- Conexões em limites hierárquicos envolvem dois tipos adicionais de conexões
  - Conexão Port-to-Port (C): Importa uma porta do componente externo para o componente interno.
    - subcomponent.port.connect(port);
    - no producer: c.put_port.connect(put_port);
  - Conexão Export-to-Export (E): Exporta um *export* para cima na hierarquia, do componente interno para o externo.
    - export.connect(subcomponent.export);
    - no consumer: put_export.connect(fifo.put_export);



Conectando Componentes

- As conexões Port-to-Port e Export-to-Export são usadas para levar os conectores a um limite hierárquico para serem acessados no nível de hierarquia imediatamente superior.

Table 1—TLM Connection Types

Connection type	connect() form
port-to-export	`comp1.port.connect(comp2.export);`
port-to-port	`subcomponent.port.connect(port);`
export-to-export	`export.connect(subcomponent.export);`

Analysis Ports

- O uvm_analysis_port (representado como um losango no monitor) é uma porta TLM especializada cuja interface consiste na função write().
- Uma port analysis_port contém uma lista de analysis_exports que estão conectadas a ela.
- Quando o componente chama analysis_port.write(), a porta de análise percorre a lista e chama o método write() de cada *export* conectado.
- Se nada estiver conectado, a chamada write() simplesmente retorna.
- Uma porta de análise pode ser conectada a zero, um ou muitos *analysis exports* (Scoreboards, coletores de cobertura, etc.).
- Como write() é uma função void, a chamada sempre será concluída no mesmo ciclo delta, independentemente de quantos componentes estejam conectados. (Não bloqueante)

Analysis Ports

```
class get_consumer_with_ap extends get_consumer;
      uvm_analysis_port #(my_trans) ap;
      function new(...);
        super.new()
        ap = new("analysis_port", this);
        ...
      endfunction
     task run_phase(uvm_phase phase);
            ...
            for(int i=0; i<10; i++)
              if(get_port.get(t)) begin
                //Do something with t.
                ap.write(t); // Write transaction.
                ...
              end
        endtask
```

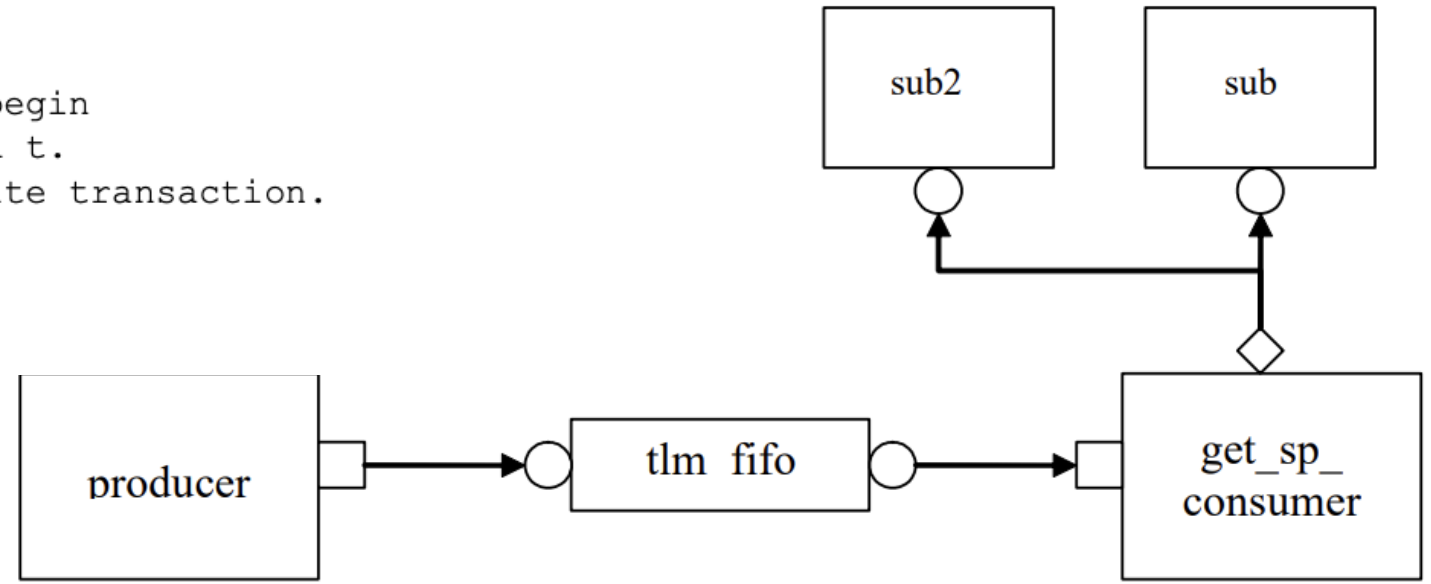


Figure 8—Analysis Communication

Analysis Exports

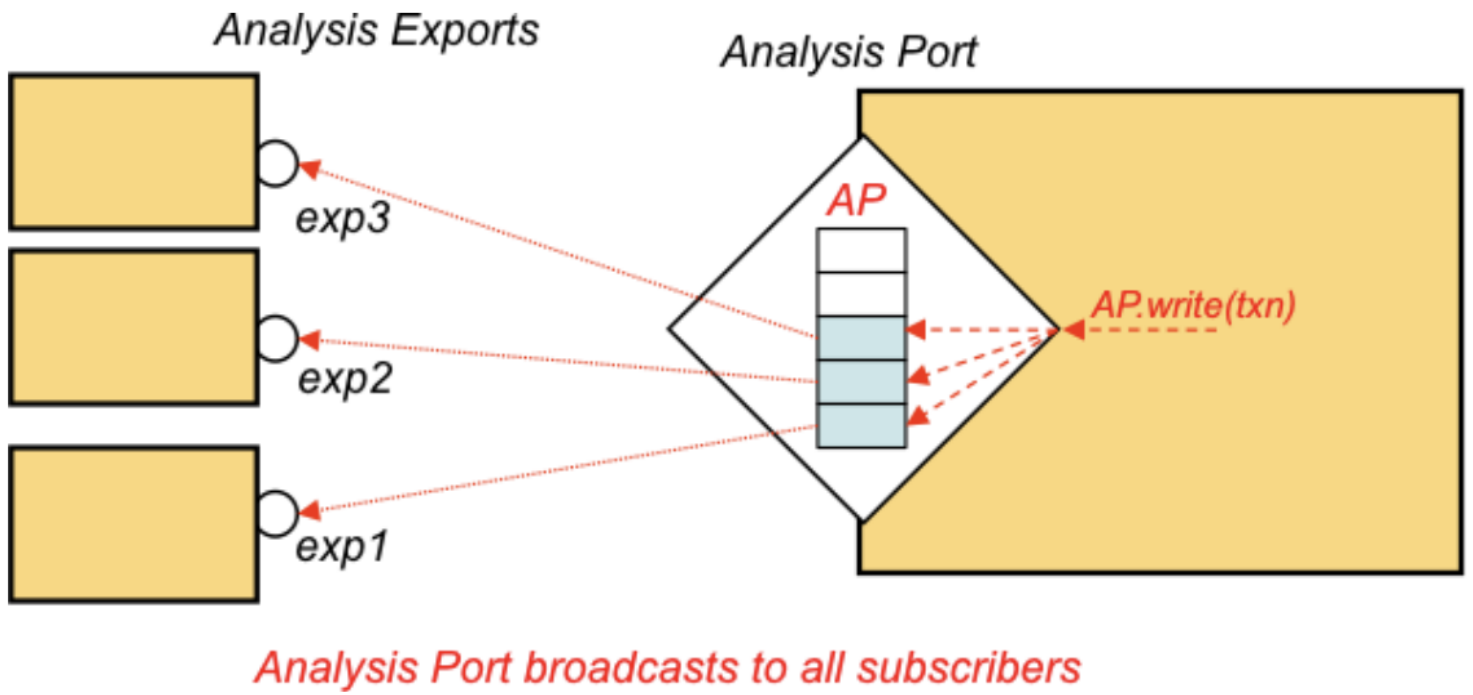
- Para que um componente possa receber transações de uma Analysis port, ele deve fornecer uma implementação da função `write()` via um `analysis_export`.
- O componente base **`uvm_subscriber`** pode ser usado para simplificar essa operação, pois ele já contém o **`analysis_export`** e o protótipo do método **`write()`**.

```
class sub1 #(type T = simple_trans) extends uvm_subscriber #(T);
    ...
      my_env env;
      function void write(T t);
        // Call desired functionality in parent.
      endfunction
endclass
```

- A classe uvm_subscriber define um método pure virtual chamado write() e o desenvolvedor deve implementar esse método na sua classe derivada para processar a transação recebida.

Conexão e Execução

- A conexão TLM entre uma porta analysis_port e um analysis export permite que o export forneça a implementação de write().
- Se múltiplos exports estiverem conectados à mesma analysis_port, a porta chamará o write() de cada export em ordem.

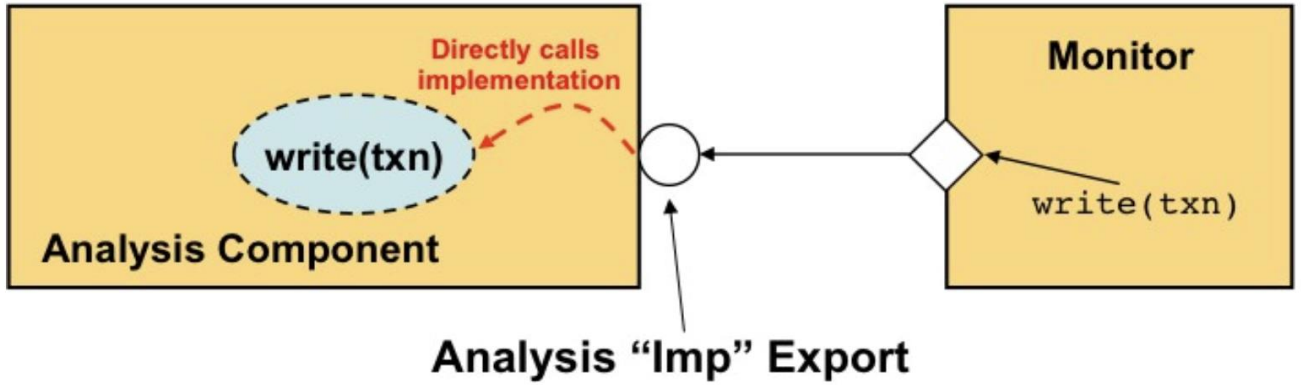


Conexão e Execução

- Quando múltiplos assinantes estão conectados, cada um recebe um ponteiro para o mesmo objeto de transação (o argumento da chamada write()).
- Cada implementação write() deve fazer uma cópia local da transação (make a local copy of the transaction), geralmente usando t.copy(), e então operar nessa cópia.
- Isso evita a corrupção do conteúdo da transação para qualquer outro assinante que possa ter recebido o mesmo ponteiro.

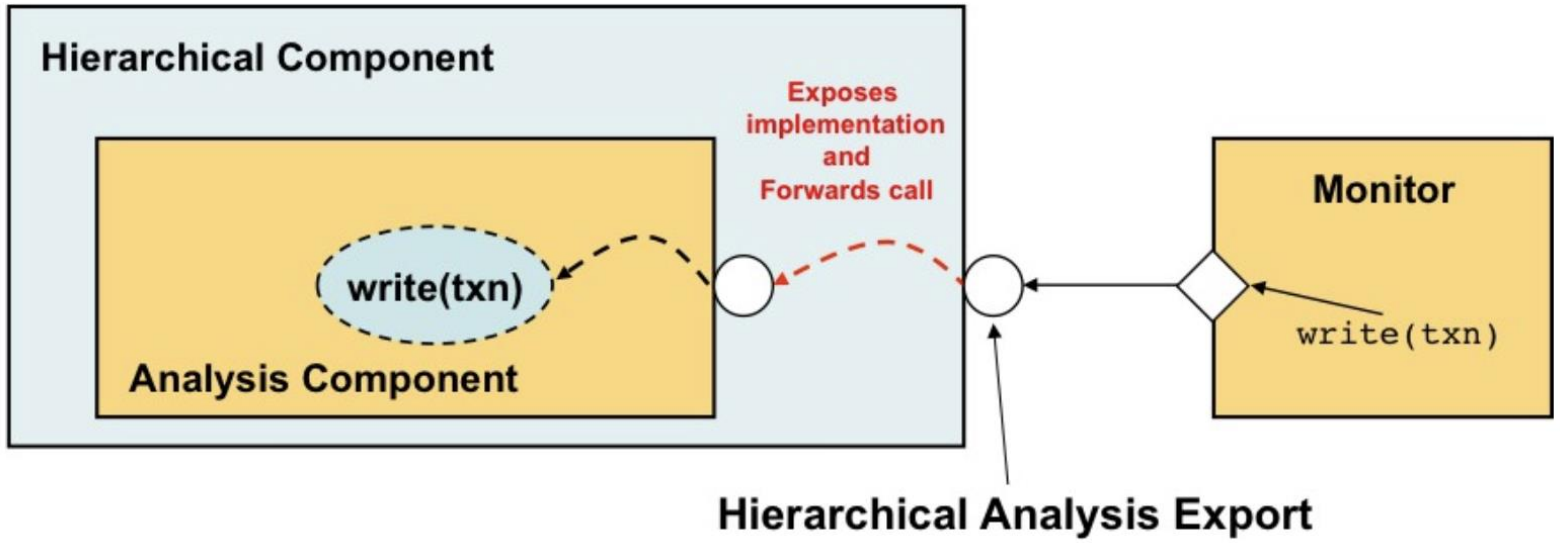
Conexão e Execução

- Assim como acontece com outros exports TLM, um analysis export existe em dois tipos: um export hierárquico ou um export "imp".
- Tanto exports hierárquicos quanto exports "imp" podem ser conectados a uma porta.
- Um export "imp" é colocado em um componente que implementa a função write() diretamente.



Conexão e Execução

- Assim como acontece com outros exports TLM, um analysis export existe em dois tipos: um export hierárquico ou um export "imp".
- Tanto exports hierárquicos quanto exports "imp" podem ser conectados a uma porta.
- Um export "imp" é colocado em um componente que implementa a função write() diretamente.



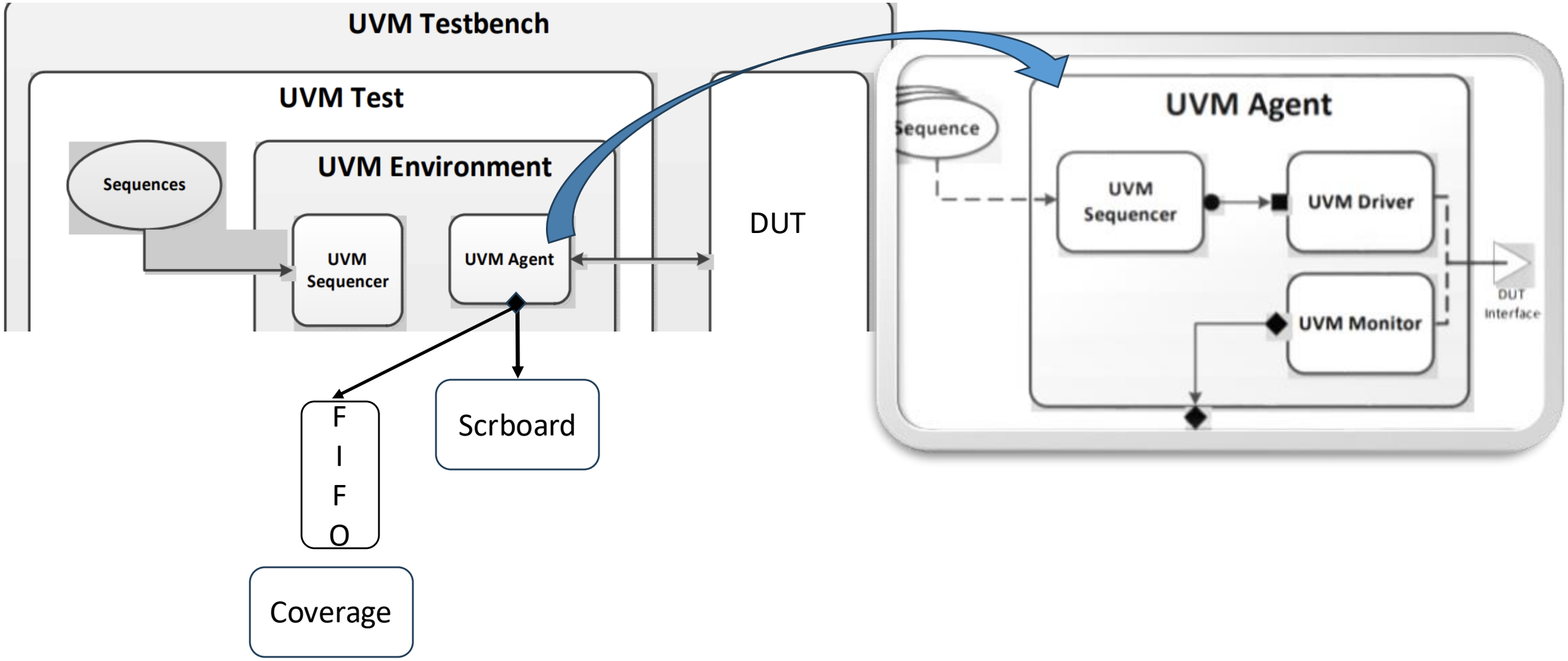
uvm_analysis_fifo

- O UVM também inclui o uvm_tlm_analysis_fifo.
- É um uvm_tlm_fifo que inclui um analysis_export.
- Permite que componentes bloqueantes tenham acesso ao fluxo de transações de análise.
- O analysis_fifo é ilimitado (*unbounded*). Isso garante que a chamada write() do monitor sempre tenha sucesso imediatamente.
- O componente de análise pode então obter as transações do analysis_fifo usando o método get() quando for conveniente, garantindo um processamento síncrono ou sequencial, se necessário.

Laboratório

- No ambiente uvm completo do modulo A, você terá que implementar as analisys port/import/export de cada modulo que tem conexão, seguindo a figura a baixo.
- Inserir uvm_analysis_fifo

UVM


UVM Testbench
UVM Test
Sequences
UVM Environment
UVM Sequencer
UVM Agent
DUT
F
I
F
O
Scrboard
Coverage
Sequence
UVM Agent
UVM Sequencer
UVM Driver
UVM Monitor
DUT Interface